

NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD



Parallel and Distributed Computing (LAB)

Lab Report: 06

Submitted to
Sir Farhad Riaz

Submitted By
Junaid Asif
(BSAI-144)

Submission Date: November 17, 2025

Task 01

Problem Statement

Write a basic MPI program that prints a message from each process along with its rank and the total number of processes.

Code

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[])
{
    MPI_Init(&argc, &argv);

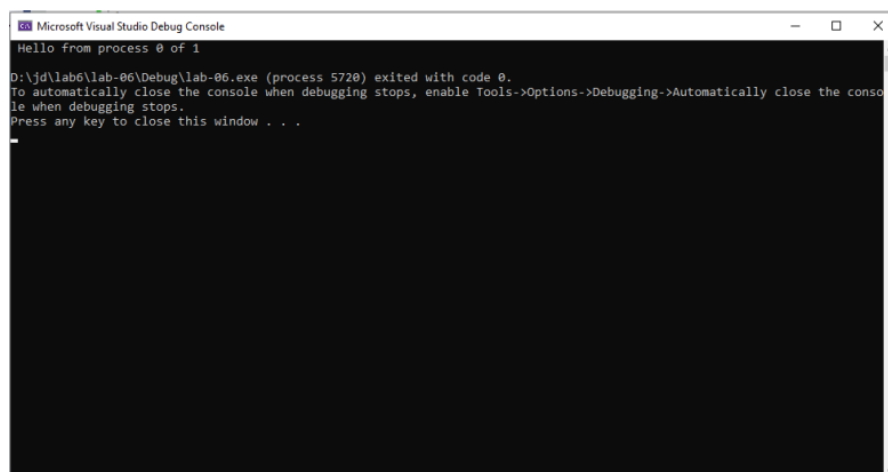
    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Process id
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Total number of processes

    std::cout << "Hello from process " << rank
              << " of " << size << std::endl;

    MPI_Finalize();
    return 0;
}
```

Output



Explanation (What, Why, How)

- **What:**
Every MPI process prints its own message containing rank and size.
- **Why:**
To verify that MPI is correctly spawning multiple processes and that each process has a unique ID (*rank*).
- **How:**
 - MPI_Init() → starts MPI environment.
 - MPI_Comm_rank() → gives each process its ID.
 - MPI_Comm_size() → gives total number of processes.
 - MPI_Finalize() → ends MPI environment.

Important Concept

- Processes execute **independently** — no communication here, only identification.

Task 02

Problem Statement

Broadcast a variable from **root process (0)** to all other processes.

Code

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[])
{
    int rank, size;
    int data;

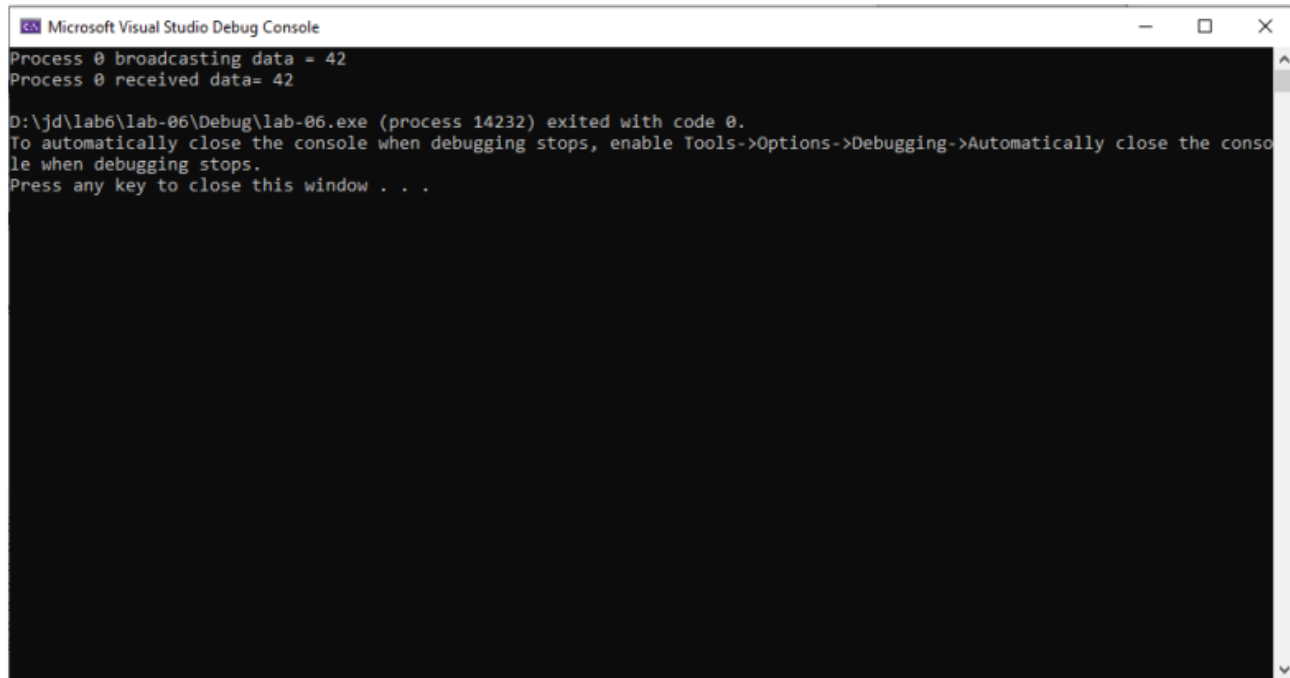
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0)
    {
        data = 42;
```

```
    printf("Process %d broadcasting data = %d\n", rank, data);  
}  
  
MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);  
  
printf("Process %d received data = %d\n", rank, data);  
  
MPI_Finalize();  
return 0;  
}
```

Output



The screenshot shows the Microsoft Visual Studio Debug Console window. The output text is as follows:

```
Process 0 broadcasting data = 42  
Process 0 received data= 42  
  
D:\jd\lab6\lab-06\Debug\lab-06.exe (process 14232) exited with code 0.  
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.  
Press any key to close this window . . .
```

Explanation

- **What:**
The root process (0) sends the same integer to all other processes.
- **Why:**
Many algorithms require **common input** or **shared initial values**. Broadcasting prevents each process from reading the file or asking the user separately.
- **How:**
 - MPI_Bcast() synchronizes all processes.
 - The root sends data, and all other processes receive it.

Why MPI_Bcast?

- Ensures all processes get identical data at the same instant.
- Reduces communication overhead compared to sending individual messages.

Task 03

Problem Statement

Each process provides a value, and MPI reduces them (sum) into a single result at the root.

Code

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[])
{
    int rank, size;
    int local_value, global_sum;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

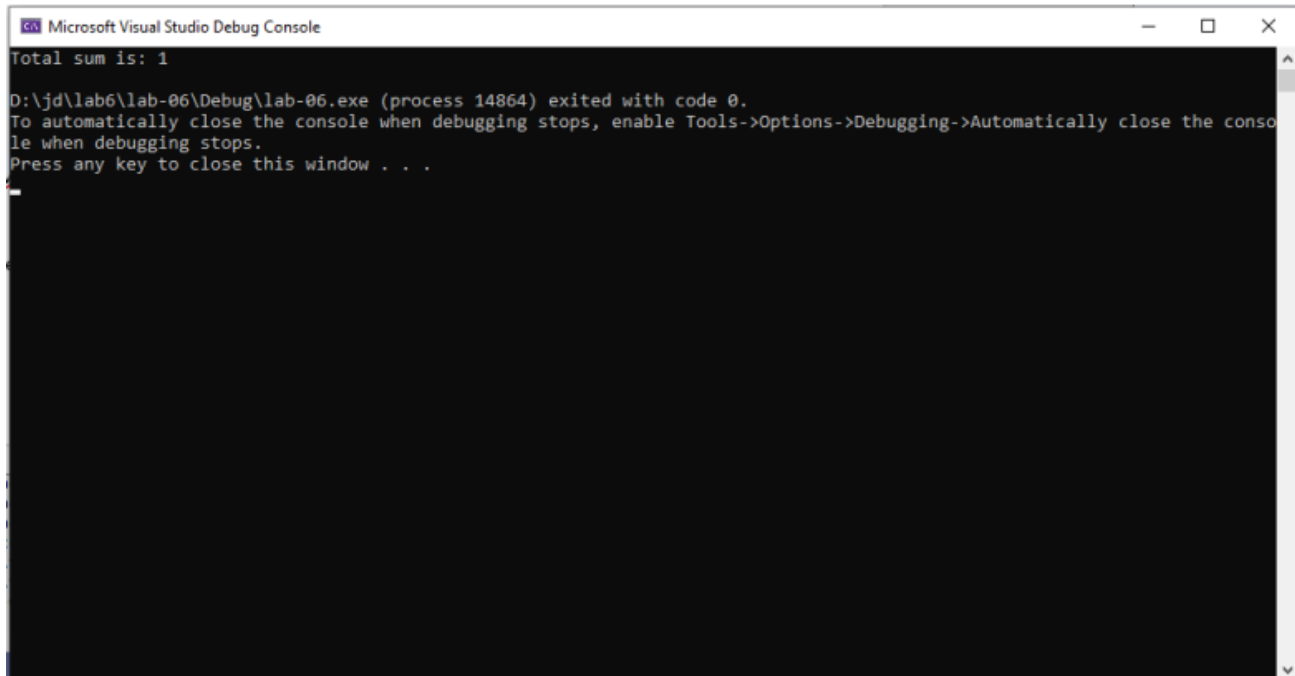
    local_value = rank + 1;

    MPI_Reduce(&local_value, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

    if (rank == 0)
    {
        printf("Total sum is: %d\n", global_sum);
    }

    MPI_Finalize();
    return 0;
}
```

Output



```
Microsoft Visual Studio Debug Console
Total sum is: 1
D:\jd\lab6\lab-06\Debug\lab-06.exe (process 14864) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:**
Each process contributes a local value, which gets summed together.
- **Why:**
Reduction operations like **sum**, **max**, **min** are extremely common in parallel processing.
- **How:**
 - Each process computes $\text{local_value} = \text{rank} + 1$.
 - `MPI_Reduce()` combines all values using `MPI_SUM`.
 - Only root (0) receives the final result.

Why MPI_Reduce?

- Efficient tree-based reduction rather than linearly adding results.
- Allows gathering combined results without sending/receiving manually.

Task 04

Problem Statement

Distribute chunks of an array from the root to all processes.

Code

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    int rank, size;
    const int root = 0;
    int* data = NULL;
    int* recvbuf;
    int n = 4;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    recvbuf = (int*)malloc(n * sizeof(int));

    if (rank == root)
    {
        data = (int*)malloc(n * size * sizeof(int));
        for (int i = 0; i < n * size; i++)
            data[i] = i + 1;
    }

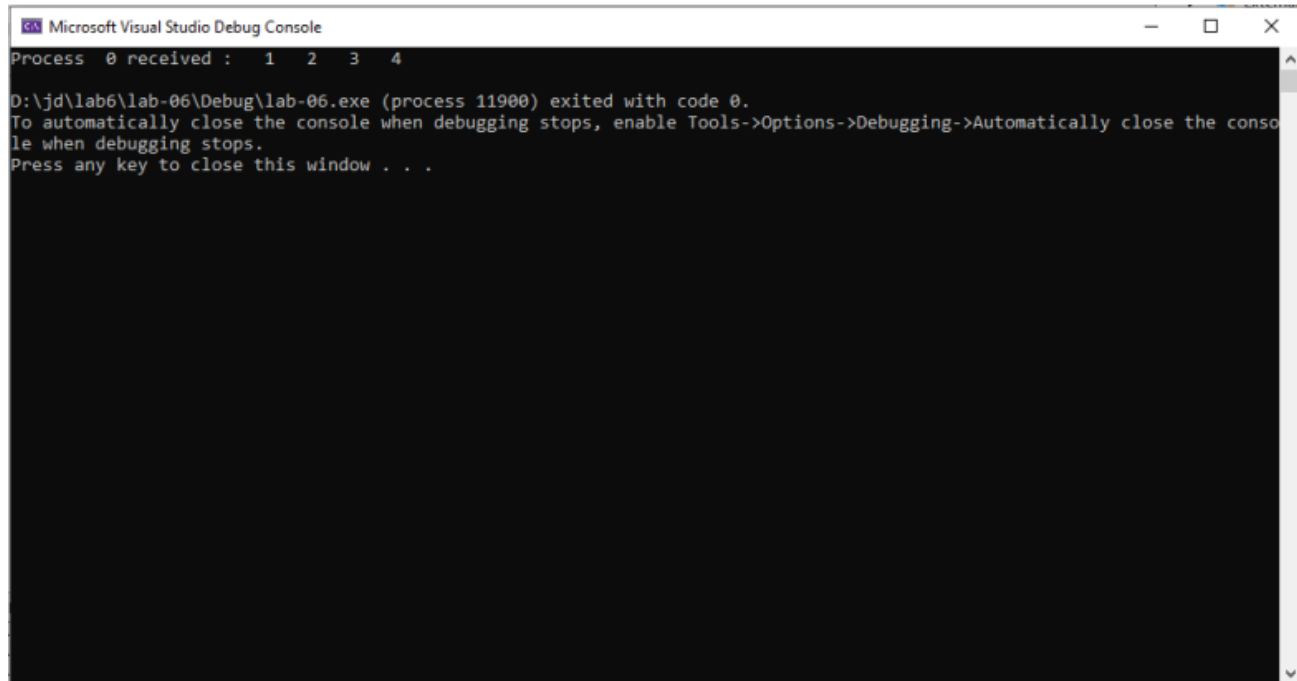
    MPI_Scatter(data, n, MPI_INT, recvbuf, n, MPI_INT, root,
MPI_COMM_WORLD);

    printf("Process %d received: ", rank);
    for (int i = 0; i < n; i++)
        printf("%d ", recvbuf[i]);
    printf("\n");

    if (rank == root) free(data);
    free(recvbuf);
}
```

```
MPI_Finalize();  
return 0;  
}
```

Output



```
Microsoft Visual Studio Debug Console  
Process 0 received : 1 2 3 4  
D:\jd\lab6\lab-06\Debug\lab-06.exe (process 11900) exited with code 0.  
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.  
Press any key to close this window . . .
```

Explanation

- **What:**
Data array is divided among processes — each gets **n elements**.
- **Why:**
In parallel computing, tasks often require dividing a dataset between processes for distributed computation.
- **How:**
 - Root creates the full data array.
 - MPI_Scatter() sends **consecutive blocks** to each process.
 - Each process receives its chunk in recvbuf.

Why MPI_Scatter?

- Eliminates need for manual point-to-point sending.
- Ensures equal data distribution.
- Extremely useful in large parallel workloads (matrix multiplication, filtering, etc).

✓ Comparative Analysis of All MPI Tasks

Feature	Task 01	Task 02	Task 03	Task 04
Main MPI Function	MPI_Comm_rank / MPI_Comm_size	MPI_Bcast	MPI_Reduce	MPI_Scatter
Type of Communication	None (Independent)	One-to-all	Many-to-one	One-to-many (split)
Data Flow	No data sharing	Root → All	All → Root	Root → Each
Purpose	Identify processes	Send same data	Combine values	Distribute data blocks
Synchronization	No	Yes (implicit)	Yes	Yes
Typical Use	Testing MPI setup	Sharing input	Summation, aggregation	Load balancing datasets

Why Each Function Is Used & Its Effect

1. MPI_Init / MPI_Finalize (Used in All Tasks)

- **Why:** Start and end MPI environment.
- **Effect:** Required for any MPI operation.

2. MPI_Comm_rank & MPI_Comm_size (Task 01)

- **Why:** Needed for process identification and distribution.
- **Effect:** Every process knows its own role.

3. MPI_Bcast (Task 02)

- **Why:** To send the same data to all processes.
- **Effect:** Removes redundant input operations; ensures same starting values.

4. MPI_Reduce (Task 03)

- **Why:** Combine all process results into one final result.
- **Effect:** Efficient parallel aggregation (parallel sum, max, min).

5. MPI_Scatter (Task 04)

- **Why:** Balance workload by splitting data among processes.
- **Effect:** Faster parallelization by distributing tasks.